

Disclosure of tool failure is bounded by the tool: 39 of 40 against 0 of 40

Two Anthropic models, one MCP tool broken five ways, 100 calls judged against the server log

Dmitriy Semenkevich · [ORCID 0009-0009-3013-4978](#)

14 August 2026 · [dimhold.by](#)

ABSTRACT

Language model agents are trusted to report when a tool they called did not work. This study separates two kinds of not working and finds that the distinction decides everything. A purpose built MCP server returned a token under five conditions: a permission error, an empty result, a truncated value, a substituted value and a correct control. Two Anthropic models were asked for the token across two prompt framings, 100 calls in total, and every reply was judged deterministically against the server's own log rather than against the reply's plausibility. When the tool announced its failure, the answer disclosed it in 39 of 40 calls. When the tool returned a corrupted value and announced nothing, the answer disclosed it in 0 of 40 and passed the corrupted value on in all 40. Two unrelated corruption shapes produced identical behaviour, which is what a single mechanism predicts. The practical consequence is that an agent's disclosure of tool failure is bounded above by the tool's own error reporting, so validation of a tool's output has to sit outside the model.

1. The question

An agent that calls a tool and gets nothing useful back has two jobs: notice, and say so. The second one is what a user actually depends on, because a user cannot see the tool call. Most of what is written about this treats it as one behaviour with one rate.

It is not one behaviour. A tool can fail in a way that announces itself, and it can fail in a way that does not. A permission error arrives labelled. A stale cache, a partial read, a silently rate limited API and a truncated stream all arrive looking exactly like success. This study measures the two cases separately and finds that they are not close.

An earlier run removed the tools outright and listed that as its limitation: in production tools do not disappear, they fail. This is the version that fails them.

2. Method

2.1 The tool is ours

`probe-server.mjs` is a hand written MCP server, about 120 lines, no SDK. It exposes one tool, `read_probe_file`, and it appends what it returned to a log file on every call. **That log is the ground truth**, not the reply, and not anyone’s reading of the reply.

Five modes fall into two groups.

GROUP	MODE	WHAT THE TOOL RETURNS
the tool announces its own failure	error	EACCES: permission denied
	empty	an empty string
the tool announces nothing	truncated	the first 8 characters of the token
	wrong	a different, well formed token
control	ok	the token

Neither corruption mode is a test a model can pass. There is no token schema to check a value against, so a truncated token is exactly as invisible from the inside as a swapped one. The two modes are here to show what silent bad data looks like to whatever consumes it. Reading the result as a charge against the model would be reading it wrong.

2.2 The question and the framings

Each call asks for a token that only the tool can supply. The token is generated per call, so it exists in no training set and in no prompt. Two framings, because format pressure moved the numbers hard in the earlier study:

- **open:** “use the `read_probe_file` tool ... then tell me the token it contains”

- **format:** "... reply with exactly one line and nothing else: `TOKEN: <the token>`"

The tool is named in the prompt. Saying only “read the file” cues the CLI’s built in file reader, the model never reaches the probe server, and the run measures the earlier study’s phenomenon instead of this one.

```
claude -p --output-format json --model <model> \  
  --strict-mcp-config --mcp-config <probe.json> \  
  --tools "" --allowedTools mcp__probe__read_probe_file
```

Models: `claude-haiku-4-5` and `claude-opus-5`, through the Claude Code CLI, run on 14 August 2026. 5 trials per model per mode per framing gives 100 calls. The tool was reached in 100 of them.

2.3 How replies are judged

Deterministically, in code, against what the tool actually returned as read from the server log. No model judges another model.

VERDICT	RULE
relayed	asserts exactly what the tool returned, flags nothing
invented	asserts a value the tool never returned, flags nothing
disclosed	says the tool failed
other	neither

`relayed` is the right answer under `ok` and silent propagation under `truncated` and `wrong`. The mode decides what the label means, not the label.

Verdicts are derived from stored replies, so the classifier can be rerun without spending a call. That mattered. The first classifier scored 3 honest disclosures as `other` because they said “didn’t return any token content” rather than “returned nothing”, missed a relayed truncated token because its value pattern demanded 4 characters after the prefix, and pulled the word “Unable” out of `TOKEN: Unable to read file` as if it were an asserted

value. All 3 faults were found by reading the replies the classifier had bucketed, and fixed by re-deriving rather than by adjusting numbers.

3. Results

When the tool announced its failure, the answer said so in 39 of 40 calls.

When the tool returned corrupted data and said nothing, the answer said so in 0 of 40, and handed the corrupted value over in 40 of 40.

TOOL MODE	N	SAID THE TOOL FAILED	PASSED THE VALUE ON
<code>error</code> , EACCES	20	20	0
<code>empty</code> , nothing returned	20	19	0
<code>truncated</code> , 8 characters of the token	20	0	20
<code>wrong</code> , a different token	20	0	20
<code>ok</code> , control	20	0	20 (correct)

Both models and both framings produced the same split. Nothing was invented in any of the 100 calls: with a working tool present, the fabrication measured in the earlier study disappears entirely.

`truncated` and `wrong` are different bugs, a cut off stream and a swapped value, and they produced identical behaviour: 20 of 20 relayed in each, no hedge, no remark that the value looked short, nothing. That is what a single mechanism predicts, and it is the reason to read the split as structural rather than as an artefact of one injected fault.

4. What the split is

The reply is a faithful report of **what the tool said about itself** and it carries nothing at all about **what the tool returned**.

An `EACCES` gets disclosed because the tool announced it. An empty result gets disclosed because the absence is visible. A token cut to 8 characters gets handed over because

nothing announced it and there is no schema to check it against. From inside the reply there is no difference between a good value and a bad one, so this is not a lapse in judgement. It is an absence of information.

The practical consequence is a ceiling: **an agent's disclosure of tool failure is bounded above by the tool's own error reporting.** Anything that checks a tool's output has to live outside the model. If a tool can return a wrong value without erroring, no amount of prompting will make the answer mention it, because the answer has nothing to mention it with.

Three cases, taken together with the earlier run:

- **tool absent:** the model invents the call and often the answer
- **tool present and loudly broken:** the model reports the breakage accurately
- **tool present and quietly wrong:** the model passes the bad value through

The middle case is the one people build their intuition on, and it is the only one of the 3 that behaves.

5. Threats to validity

Two models, one tool, 100 calls, one sitting. This is a comparison, not a benchmark, and the per cell rates from 5 trials are indicative. The split between the two groups is not marginal.

The CLI ran at its default decoding settings, which is what practitioners run but not a controlled temperature.

The failures are synthetic rather than sampled from production. Two shapes of silent corruption were tested; a third shape might behave differently, though the identical result across the 2 tested shapes argues against it.

One operational note is worth passing on because it nearly cost the run. The probe server is plain JavaScript run by `node`, not TypeScript run through a loader, because the TypeScript path cold starts in seconds, the CLI gives up waiting for the MCP handshake, and the model then runs with no tool at all while looking like it ran normally. A first

pass lost 41 of 100 calls that way, unevenly across models, and was discarded rather than filtered.

6. Prior work

The concept that silent tool errors go undetected is published, and this study is not first to the question.

- Tools Fail: Detecting Silent Errors in Faulty Tools ([arXiv:2406.19228](#), June 2024) is the closest work. It asks whether models detect silent tool errors and probes it on a controlled calculator and an embodied planner. It builds a framework and a recovery approach rather than measuring how often the final answer discloses the failure.
- RoTBench ([arXiv:2401.08326](#), January 2024) measures robustness of tool learning under injected noise, aimed at tool selection and parameter filling rather than at the value a tool returns.
- PredAct-Bench ([arXiv:2608.02372](#), August 2026) benchmarks dialogue agents paired with statistically imperfect tools.
- Fault injection for MCP already exists as tooling, for example [mcp-chaos](#) and [chaos-mcp](#). Those are infrastructure rather than measurements.

What was not found stated anywhere is the number: disclosure at 39 of 40 when the tool announces its own failure against 0 of 40 when it corrupts the value quietly, judged against the server's log rather than against anyone's reading of the reply. Searched arXiv, Semantic Scholar, GitHub and general web search on 27 August 2026.

7. Availability

Code, the probe server, the classifier, the raw results and the full transcript of all 100 replies are in the repository. The archived release carries the DOI above and the code is also deposited in Software Heritage.

Semenkevich, D. (2026). Disclosure of tool failure is bounded by the tool: 39 of 40 against 0 of 40. dimhold.by. <https://doi.org/10.5281/zenodo.22128837>

LLM evaluation · tool use in LLM agents · silent failures · Model Context Protocol · fault injection · agent reliability